

Lema

Dado un autómata finito que permite transiciones etiquetadas con λ , se puede construir otro autómata equivalente sin transiciones etiquetadas con λ .

Demostración.

Vamos a dar simplemente un esquema de esta demostración. Dado un AF M puede obtenerse otro M' no determinista equivalente y sin transiciones λ a través de las siguientes transformaciones:

1. Agregar a M' una λ -transición de un estado s_i a otro s_j cuando en M se llegaba desde s_i a s_j por medio de más de una transición λ .
2. Poner como estados aceptadores todos aquellos estados de M que por λ permiten llegar a un estado aceptador.
3. por cada par de transiciones $\delta(s_i, \lambda) = s_j$, $\delta(s_j, a) = s_k$ agregar $\delta(s_i, a) = s_k$.
4. Sacar del autómata obtenido hasta el momento las λ -transiciones $\delta(s_i, \lambda) = s_j$ para todo $s_i, s_j \in S$.



Autómata Finito Reconocedor (con λ)

Definición

Un *autómata finito reconocedor* (AFR) es una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

- S es el conjunto de estados, $S \neq \emptyset$.
- Σ es el alfabeto de entrada.
- $\delta: S \times (\Sigma \cup \{\lambda\}) \rightarrow S$ es la función de próximo estado o de transición.
- s_0 es el estado inicial, $s_0 \in S$.
- F es el conjunto de estados finales o aceptadores $F \subseteq S$.

$$\delta(s_i, \lambda) = s_j$$

Esta será la definición que utilizaremos de aquí en más para los AFR.



Definición

Sea Σ un alfabeto, se dice que un AFR $M = (S, \Sigma, \delta, s_0, F)$ acepta (o reconoce) una cadena $w \in \Sigma^*$ si, partiendo de $\delta(s_0, a)$ alcanzamos un estado s_f donde $s_f \in F$.

Definición

Sea $M = (S, \Sigma, \delta, s_0, F)$ un AFR, el lenguaje reconocido por el autómata M es

$L(M) = \{w \mid w \text{ es aceptada o reconocida por } M\}$.

w

z

$w \in \Sigma^*$

$z \in \Delta^*$

Autómatas Finitos Traductores

$$M = (S, \Sigma, \delta, s_0, F)$$

Definición

Un *autómata finito traductor de Moore* es una séxtupla, $M = (S, \Sigma, Z, \delta, s_0, f_0)$ donde

→ alfabeto de salida

- $f_0: S \rightarrow Z$. Aquí Z es el alfabeto de salida que varia según el problema planteado. El resto de los elementos se interpretan como en el caso del AFR.

→ tenga en cuenta que la salida está en el nodo. Luego, la función de salida se indicará en la tabla de estados con una columna extra:

δ	Σ	Z
estados	entradas	salida
$\rightarrow s_0$	...	z_0
s_1	...	z_1
...	...	...

→ s_i/z_i

De esta manera el AFT de Moore imprime la salida cuando **alcanza** el estado.

Ejemplo AFT de Moore

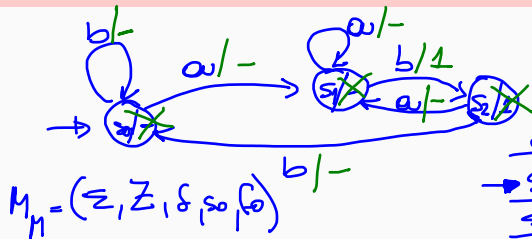
$$L = \{ w \in \{a,b\}^* \mid w_1 a a a b b w \in L \}$$

1

$w_2 \in L$? $w_4 = a a a$
 $w_3 = a b a b a b$

Ejemplo

Tomemos el problema de contar las apariciones de la cadena ab en cadenas de $\{a,b\}^*$.



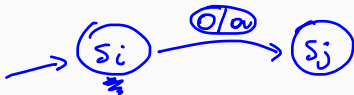
$a b a b$

1 1

δ	a	b	f_0
s_0	$s_1/-$	$s_0/-$	-
s_1	$s_1/1$	$s_2/1$	-
s_2	$s_1/-$	$s_0/0$	1

$\Sigma = \{a, b\}$
 $Z = \{1\}$

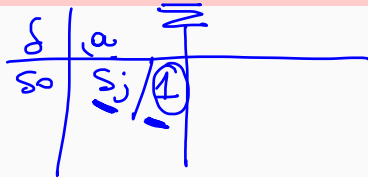
AF Traductor de Mealy



Definición

Un *autómata finito traductor de Mealy* es una sextupla $M = (S, \Sigma, Z, \delta, s_0, f_0)$ donde $f_0: S \times \Sigma \rightarrow Z$ y los demás elementos se interpretan como en el caso del AFT de Moore.

Las salidas se colocan en los arcos.



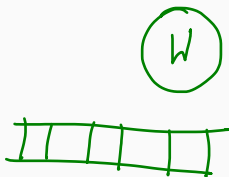
.

Ejemplo

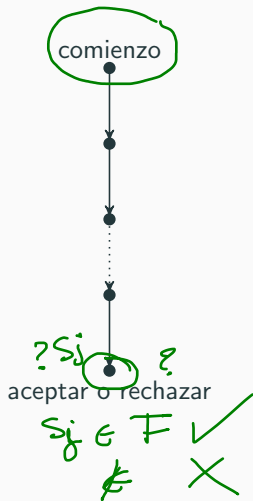
Cómo quedaría el AFT de Mealy para el problema de contar las apariciones de la cadena ab en cadenas de $\{a, b\}^*$.

Autómatas Finitos No Deterministas

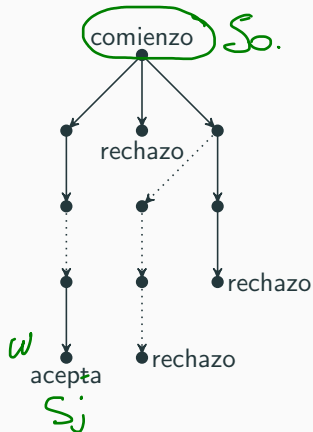
Autómatas Finitos No Deterministas



AFRD.
Determinismo



AFND
no Determinismo

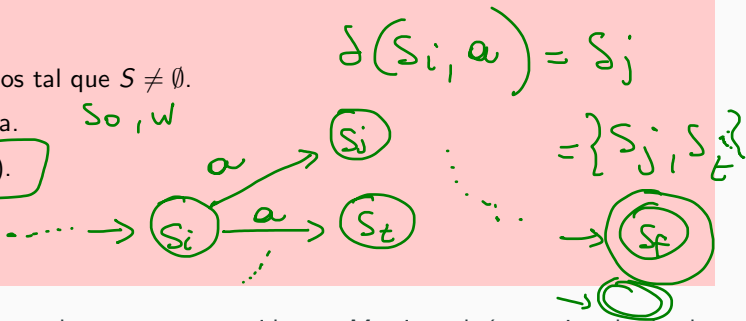


Autómatas Finitos No Deterministas

Definición

Un autómata finito reconocedor no determinista (AFND) es definido mediante una quintupla $M = (S, \Sigma, \delta, s_0, F)$ donde:

- S es un conjunto de estados tal que $S \neq \emptyset$.
- Σ es el alfabeto de entrada.
- $\delta : S \times (\Sigma \cup \{\lambda\}) \rightarrow P(S)$.
- $s_0 \in S$
- $F \subseteq S$



Intuitivamente, decimos que una cadena w es reconocida por M existe algún camino desde el estado inicial a un estado final, siguiendo la función δ .

Ejemplo AFND

Ejemplo

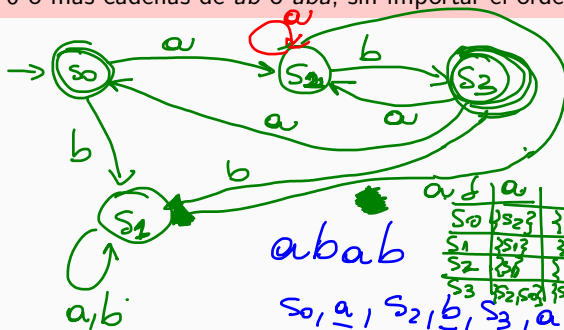
Diseñar un autómata finito no determinista que acepte el lenguaje de las cadenas formadas por concatenación de 0 o más cadenas de ab o aba , sin importar el orden.

?
 $\lambda \in L$
 $ab \in L \checkmark$
 $abba \in L \times$

$ababa \checkmark$

$bbab \times$

ninguna cadena que
 comience con $b \in L$.
 . 2 b's pegados no pertenecen.



$aaab \in L(M)$
 $aaaab \in L$
 $L(M) \neq L$

$M =$

	a	b
S_0	$\{S_2\}$	$\{S_1\}$
S_1	$\{S_2\}$	$\{S_0\}$
S_2	$\{S_3\}$	$\{S_0\}$
S_3	$\{S_2, S_0\}$	$\{S_1\}$

$S = \{S_0, S_1, S_2, S_3\}$
 $F = \{S_0, S_3\}$

$abab$

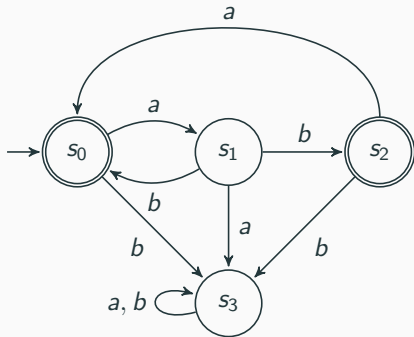
$s_0, \underline{a}, s_2, \underline{b}, s_3, \underline{a}, s_2, \underline{b}, s_3$

$ababab \checkmark$ $s_0, \underline{a}, s_2, \underline{b}, s_3, \underline{a}, s_0, \underline{a}, s_2, \underline{b}, s_3, \underline{a}, s_0$
 $s_0, \underline{a}, s_2, \underline{b}, s_3, \underline{a}, s_2, \underline{a}, s_1 \times$

Ejemplo AFND

Ejemplo

Diseñar un autómata finito no determinista que acepte el lenguaje de las cadenas formadas por concatenación de 0 o más cadenas de ab o aba , sin importar el orden.



Equivalencia entre AFD y AFND

Vamos a querer establecer y demostrar que el poder expresivo de un AFD es equivalente al de un AFND.

Teorema

Sea L un lenguaje aceptado por un autómata finito no determinista, entonces existe un autómata finito determinista que también acepta L .

$$M = (S, \Sigma, \delta, s_0, F)$$

↑

$$\delta: S \times \Sigma \rightarrow P(S)$$

Equivalencia entre AFD y AFND demostración

Sea $M = (S, \Sigma, \delta, s_0, F)$ un AFND que acepta L . Para probar el teorema debemos probar: (a) que podemos construir un AFD M' a partir de M y (b) que M' acepta el mismo lenguaje que M .

(a) AFD a partir un AFND

- $S' \subseteq P(S)$ Los estados de M' serán identificados con **subconjuntos** del conjunto de estados de M . Para esto notaremos un elemento de S' como $\langle s_1, s_2, \dots, s_i \rangle$ donde $\{s_1, s_2, \dots, s_i\} \subseteq S$.
- $F' = \{s'_i \in S' \mid s'_i \supseteq \{s_j\} \text{ y } s_j \in F\}$. F' es el conjunto de todos los estados de S' que contengan por lo menos un elemento en F (es decir que contengan un **estado aceptador** de M).
- $s'_0 = \langle s_0 \rangle = \{s_0\}$. s'_0 será el **conjunto** que contiene únicamente al estado inicial de M .
- La función δ' es una extensión de la función δ a conjuntos:
 - $\delta'(\langle s_1, s_2, \dots, s_i \rangle, a) = \langle p_1, p_2, \dots, p_j \rangle \Leftrightarrow \delta(\{s_1, s_2, \dots, s_i\}, a) = \{p_1, p_2, \dots, p_j\}$. Dicho de otra manera δ' aplicada a un elemento $\langle \bullet \rangle$ de S' se computa aplicando δ a **cada estado** de S representado en $\langle \bullet \rangle$. Como δ' se aplica a un conjunto de estados es natural que el resultado también sea un conjunto de estados. Luego $\delta(\{s_1, s_2, \dots, s_i\}, a) = \bigcup_{k=1}^i (\delta(s_k, a))$. Es decir, si tomamos $\langle s_1, s_2, \dots, s_i \rangle$ aplicando δ a cada s_k , $1 \leq k \leq i$, y tomando la unión obtenemos los estados de llegada $p_1, p_2, \dots, p_j$. Este conjunto $\langle p_1, p_2, \dots, p_j \rangle$ es entonces **otro estado** de M' , y es el resultado de aplicar $\delta'(\langle s_1, s_2, \dots, s_i \rangle, a)$.
 - Tenga en cuenta que mientras el resultado de la función δ es un **conjunto de estados** de M , el resultado de δ' es un **solo estado** denotado como $\langle p_1, p_2, \dots, p_j \rangle$, luego M' es determinista.

Equivalencia entre AFD y AFND demostración

(b) Ahora falta probar que $\delta'(s'_0, w) = \langle q_1, q_2, \dots, q_i \rangle$ si y solo si $\delta(s_0, w) = \{q_1, q_2, \dots, q_i\}$ para toda cadena w . Lo probaremos por inducción sobre la longitud de la cadena w .

- **Caso Base:** Si $|w| = 0$, entonces $w = \lambda$ y la demostración es trivial ya que $s'_0 = \langle s_0 \rangle$.
- **Hipótesis Inductiva:** Supongamos que es válido para w con $|w| = l$, es decir, $\delta'(s'_0, w) = \langle p_1, p_2, \dots, p_n \rangle$ si y solo si $\delta(s_0, w) = \{p_1, p_2, \dots, p_n\}$, sea $a \in \Sigma$, debemos probar que vale para wa con $|wa| = l + 1$.
- **Paso Inductivo:** Por definición

$$\delta'(s'_0, wa) = \delta'(\delta'(s'_0, w), a) \quad (1)$$

Por hipótesis:

$$\delta'(s'_0, w) = \langle p_1, p_2, \dots, p_n \rangle \Leftrightarrow \delta(s_0, w) = \{p_1, p_2, \dots, p_n\}$$

reemplazando en la Ecuación 1 y por definición de δ' tenemos que:

$$\delta'(\langle p_1, p_2, \dots, p_n \rangle, a) = \langle q_1, q_2, \dots, q_k \rangle \Leftrightarrow \delta(\{p_1, p_2, \dots, p_n\}, a) = \{q_1, q_2, \dots, q_k\}$$

Entonces,

$$\delta'(s'_0, wa) = \langle q_1, q_2, \dots, q_k \rangle \Leftrightarrow \delta(s_0, wa) = \{q_1, q_2, \dots, q_k\} \quad (2)$$

En particular $\delta'(s'_0, w)$ está en F' (un estado final de M') cuando $\delta(s_0, w)$ tiene un estado perteneciente a F . Luego una cadena es aceptada por M' si era aceptada por M .

Ejemplo AFD a partir de un AFND

Ejemplo

Sea un AFND $M = (S, \Sigma, \delta, s_0, F)$ donde $S = \{s_0, s_1, s_2\}$, $\Sigma = \{0, 1\}$, $F = \{s_1\}$ y δ definido por la siguiente tabla:

δ	0	1
$\rightarrow s_0$	$\{s_0, s_1\}$	$\{s_1\}$
$*s_1$	$\{s_2\}$	$\{s_0, s_1\}$
s_2	$\{s_2\}$	$\{s_2\}$

Obtenga el AFD M' que reconoce el mismo lenguaje utilizando el teorema anterior.

Homomorfismos Sobre Autómatas Finitos

Definición

Sean $M = (\underline{S}, \Sigma, s_0, \delta, \underline{f_0})$ y $M' = (S', \Sigma', s'_0, \delta', \underline{f'_0})$ dos AFD reconocedores. Un *homomorfismo* g del autómata finito $\underline{M}$ a $\underline{M'}$ es una función $g : \underline{S} \rightarrow S'$ tal que para cualquier $a \in \Sigma$ y $s_i \in \underline{S}$:

$\mathbb{F}$

$\mathbb{F}'$

$$\underline{g}(s_0) = s'_0 \quad (3)$$

$$\underline{g}(\delta(s_i, a)) = \delta'(g(s_i), a) \quad (4)$$

$$\underline{f_0}(s_i) = f'_0(g(s_i)) \quad (5)$$

Ejemplo Homomorfismo

Ejemplo

Tomaremos $M = (S, \Sigma, s_0, \delta, f_0)$ y $M' = (S', \Sigma', s_a, \delta', f'_0)$ dos AFD definidos de la siguiente manera:

$$\Sigma = \Sigma' = \{0, 1\}$$

$$M = (S, \Sigma, s_0, \delta, f_0)$$

$$S = \{s_0, s_1, s_2\}$$

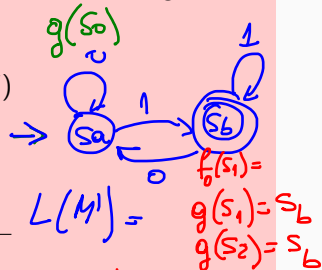
$$\Sigma = \{0, 1\}$$

δ	0	1	f_0
$\rightarrow s_0$	s_0	s_1	0
$*s_1$	s_0	s_2	1
$*s_2$	s_0	s_1	1

$$M' = (S', \Sigma', s_a, \delta', f'_0)$$

$$S' = \{s_a, s_b\}$$

δ'	0	1	f'_0
$\rightarrow s_a$	s_a	s_b	0
$*s_b$	s_a	s_b	1



$$L(M') =$$

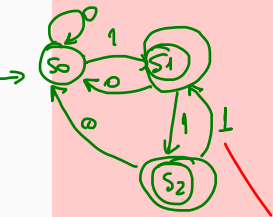
$$g(s_1) = s_b$$

$$g(s_2) = s_b$$

$$g(s_0) = s_a$$

$$g(\delta(s_1, 0)) = \delta'(g(s_1), 0)$$

$$g(s_0) = \delta'(s_b, 0) = s_a$$



$$L(M) = \{w \in \{0, 1\}^* \text{ que terminan con } 1\}$$

Definir $g: S \rightarrow S'$ a fin de obtener nuestro homomorfismo.

$$g(\delta(s_1, 1)) = \delta'(g(s_1), 1) = s_b$$

$$g(s_2) = \delta'(s_b, 1) = s_b$$

$$g(\delta(s_2, 0)) = \delta'(g(s_2), 0) = s_a$$

$$g(\delta(s_2, 1)) = \delta'(g(s_2), 1) = s_b$$

Definición (ρ)

Sea g un homomorfismo de un autómata $M = (S, \Sigma, s_0, \delta, f_0)$ en un autómata $M' = (S', \Sigma, s'_0, \delta', f'_0)$, se define la relación binaria ρ sobre S , de la siguiente manera:

$$s_i \rho s_j \text{ si y solo si } g(s_i) = g(s_j)$$

Ejemplo

Considerando los autómatas M y M' del ejemplo anterior el homomorfismo g determina en M dos clases $[s_0]$ y $[s_1]$ con $s_1 \rho s_2$ ya que $\underline{g(s_1)} = \underline{g(s_2)} = s_b$ luego $[s_1] = \{s_1, s_2\}$.

Luego, dos estados de M están relacionados por ρ cuando por el homomorfismo tienen asociado el mismo estado en M' . ρ particiona a S en clases de equivalencia. Trataremos a las **clases de estados** como **estados** de una máquina a la que denominaremos *máquina cociente* M/g .



Definición (Autómata Cociente)

Sean $M = (S, \Sigma, s_0, \delta, f_0)$ y $M' = (S', \Sigma, s'_0, \delta', f'_0)$ dos AFR y g un homomorfismo definido sobre ellos. Un *Autómata Cociente* $M/g = (S'', \Sigma, [s_0], \delta'', f'')$ es un AFR tal que:

- S'' es el conjunto de estados de M/g tal que cada uno es una clase de equivalencia $[s]$ de estados de M determinada por g . Todos los elementos de $[s]$ tienen el mismo estado M' y la misma salida.
- Σ es el alfabeto de entrada para M y M' .
- $[s_0]$ es el estado inicial de M/g y corresponde a la clase del estado inicial de M .
- f''_0 es la función de salida.
- δ'' es la función próximo estado definida mediante $\delta''([s_j], a) = [\delta(s_j, a)]$. Es decir, δ'' mapea bajo g el mismo estado de M' a todos los estados de s_n con idéntica entrada a .

Definición (Continuación Autómata Cociente)

A fin de entender mejor M/g analicemos su función próximo estado δ'' . Consideremos un estado $[s_j]$ de M/g y una entrada a . Primero, tomemos en M su función próximo estado $\delta(s_j, a)$. En M/g el próximo estado se mapea $[s_j]$ bajo a en la clase $[\delta(s_j, a)]$. Debemos verificar entonces que δ'' está correctamente definida, que su resultado sea independiente del representante de clase elegido.

Sea $s_n \in [s_j]$ entonces:

$$s_n \rho s_j \text{ si y solo si } g(s_n) = g(s_j) \quad (6)$$

Por la definición de homomorfismo g

$$g(\delta(s_j, a)) = \delta''(g(s_j), a) \quad (7)$$

y

$$g(\delta(s_n, a)) = \delta''(g(s_n), a) \quad (8)$$

luego reemplazo por la Ecuación 6 en la Ecuación 7

$$g(\delta(s_j, a)) \stackrel{7}{=} \delta''(g(s_j), a) \stackrel{6}{=} g(\delta(s_n, a)) \stackrel{8}{=} \delta''(g(s_n), a)$$

Ejemplo Autómata Cociente

M/g

Ejemplo

Vamos a ver otra vez el ejemplo de la sección anterior y armaremos M/g . El conjunto de estados $S'' = \{[s_0], [s_1]\}$ donde $[s_0] = \{s_0\}$ y $[s_1] = \{s_1, s_2\}$ (observe que s_0, s_1, s_2 son estados de M). La función de transición y de aceptación se definen de la siguiente manera:

δ'	0	1
$\rightarrow [s_0]$	$[s_0]$	$[s_1]$
$* [s_1]$	$[s_0]$	$[s_1]$

M'

¿Qué forma tendrá el digrafo para M/g ? Dibújelo. ¿Puede decir algo con respecto a M/g ?

isomorfismo

$M/g.$

M'

dominio

Definición (Isomorfismo)

Se dice que h es un isomorfismo (admite inverso) de un AFR $M'' = (S'', \Sigma, s_0'', \delta'', f_0'')$ en otro AFR $M' = (S', \Sigma, s_0', \delta', f_0')$ si h es una función $h : S'' \rightarrow S'$ tal que:

- h es un homomorfismo entre M'' y M'
- h es una biyección. Es decir, es inyectiva a elementos distintos del dominio (S'') le corresponden elementos distintos del codominio (S') y es sobreyectiva, aplicada a todo el codominio S' (cada elemento de S' es la imagen de como mínimo un elemento en S'')

Teorema

Sea g un homomorfismo sobreyectivo de M a M' (ambos AFRs), entonces la imagen homomorfa de M , M' es isomorfa al autómata finito cociente M/g .

Minimización de Autómatas Finitos (Teorema de Myhill-Nerode)



Definición (Estados inalcanzables)

Dado un autómata finito $M = (S, \Sigma, s_0, \delta, f_0)$ y un estado $s_i \in S$ tal que $s_i \neq s_0$ diremos que s_i es un estado inalcanzable de M si no existe cadena $w \in \Sigma^*$ tal que $\delta(s_0, w) = s_i$.

Definición (Estados Equivalentes)

Dos estados s_i, s_j de un AF M son *equivalentes* si para toda cadena de entrada $w \in \Sigma^*$, $\delta(s_i, w) = s_k$ y $\delta(s_j, w) = s_l$, donde $f_0(s_k) = f_0(s_l)$.

Minimización de Automatas Finitos

1011101

s_k

s_l

Definición (k-equivalencia)

Dos estados s_i, s_j de un AF M son k-equivalentes si para toda cadena de entrada $w \in \Sigma^*$, con $|w| \leq k$ $\delta(s_i, w) = s_k$ y $\delta(s_j, w) = s_l$, donde $f_0(s_k) = f_0(s_l)$.

Definición

Sean R y R' dos relaciones de equivalencia definidas sobre S diremos que R *refina* a R' , si siempre que $s_i R s_j$ entonces $s_i R' s_j$ para todo $s_i, s_j \in S$.

Minimización de Autómatas Finitos

1: **function** MINAF(M): M Minimizado

Require: un AF $M = (S, \Sigma, \delta, s_0, F)$ determinista completo

Ensure: el AF M minimizado

2: $T = S -$ los estados inalcanzables de M

3: T se particiona en dos clases por los estados *0-equivalentes*: un conjunto de estados aceptadores y otro de estados no aceptadores.

4: $k = 0$

5: **repeat**

6: Armamos las clases *k+1-equivalentes* refinando las *k-equivalentes*. s_i, s_j son *k+1-equivalentes* $\Leftrightarrow \delta(s_i, a), \delta(s_j, a)$ son *k-equivalentes* para toda $a \in \Sigma$.

7: $k++$

8: **until** las clases *k+1-equivalentes* sean iguales a las *k-equivalentes*

9: Con las clases *k+1-equivalentes* definimos M/g

10: **return** M/g

11: **end function**